Block 4 – Cloud Computing

S2: Cloud Computing & Cloud Architecture

Arquitectura de Xarxes

Universitat Pompeu Fabra

upf.

Outline

upf.

If We Can Virtualize Everything... Why Not Sell It?

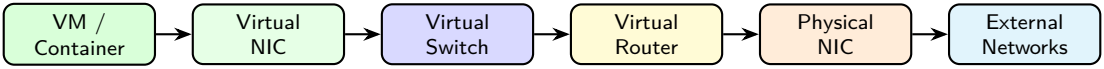
Virtualization is the technology.
Cloud computing is the business model.

Learning objectives:

1. Understand cloud computing's definition and essential characteristics
2. Distinguish IaaS (Infrastructure as a Service), PaaS (Platform as a Service), and SaaS (Software as a Service)
3. Understand core cloud architecture patterns: regions, availability zones, and virtual networks
4. Design a cloud virtual network with subnets, security controls, and network services

upf.

Recap – What We Virtualized (Session 1)



- **Compute**: VMs share physical hardware via hypervisors; containers share the host kernel directly
- **Networking**: Virtual NICs, Switches, and Routers replicate the physical stack in software
- **Isolation**: VLANs and overlay networks separate tenants on shared infrastructure

The Leap to Cloud

- Virtualization = technology (**how** you run multiple workloads)
- Cloud = business model (**what** you sell to customers)

Virtualization	**Cloud Computing**
Run multiple VMs/containers on one host	Offer compute as a **service**
Internal IT optimization	External/internal consumption
Manual provisioning possible	**Automated, self-service**
You own the hardware	Provider owns the hardware

Key difference: **automation and self-service** (no phone calls, no tickets, no waiting).

upf.

Discussion: From Virtualization to Cloud

Your company already runs VMs on its own servers.
Why would you move to the cloud instead of
keeping everything in-house?

Hints: think about CapEx vs OpEx, scaling speed, and who manages what.

Outline

upf.

Cloud Computing – Definition (NIST)

*A model for enabling ubiquitous, convenient, **on-demand network access** to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort.*

- **Ubiquitous**: anywhere, any device
- **On-demand**: no tickets, provision instantly
- **Shared pool**: many tenants, same hardware
- **Configurable**: choose CPU, RAM, storage, network
- **Rapidly provisioned**: up and running in seconds
- **Minimal management**: provider handles the hardware

Source: NIST SP 800-145 (2011). Still the standard definition used industry-wide.

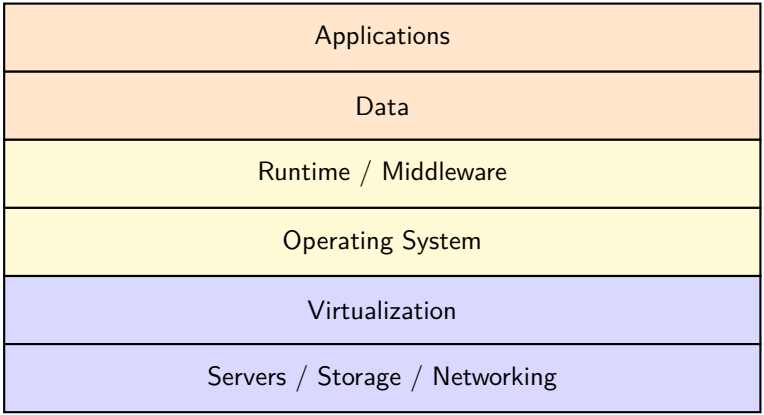
upf.

Five Essential Characteristics

Characteristic	What it means
On-demand self-service	Provision resources via portal/API, no human interaction
Broad network access	Available from any device over the network
Resource pooling	Shared infrastructure, multi-tenant
Rapid elasticity	Scale up/down automatically (e.g. Black Friday $\rightarrow 10\times$)
Measured service	Pay only for what you use (CPU-hours, GB stored)

- **All five** must be present to qualify as "cloud computing"

Cloud Service Models – The Stack



IaaS, PaaS, SaaS, CaaS – In Practice

IaaS (e.g. AWS EC2, Azure VMs, Google Compute Engine):

- Provider gives you VMs, storage, networks. You manage everything else
- "I want a Linux server in Frankfurt, 4 CPUs, 16 GB RAM, right now"

PaaS (e.g. Google App Engine, AWS Elastic Beanstalk, Heroku):

- Provider gives you runtime + tools. You only manage your code and data
- "I wrote a Python web app. Just run it, I do not care about servers"

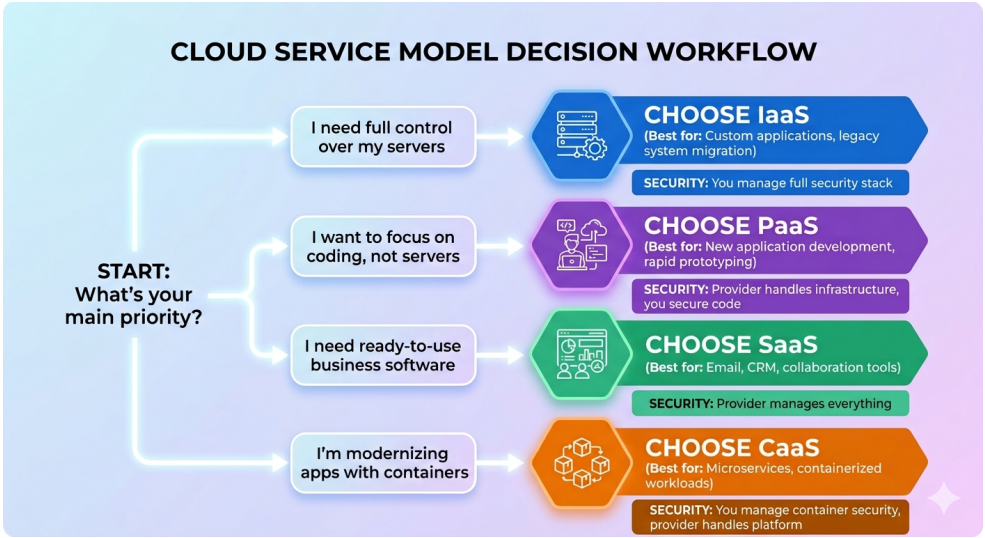
CaaS (Container as a Service) (e.g. AWS EKS, Azure AKS, Google GKE):

- Provider gives you a managed container orchestration platform (Kubernetes)
- "I have containers. Just run and scale them, I do not care about the cluster"

SaaS (e.g. Google Workspace, Microsoft 365, Slack, Zoom):

- Provider gives you a complete application. You manage your data only

Service Models – Who Manages What?



Discussion: Cloud Fundamentals

A university wants to offer a web-based tool
for students to submit assignments.
Should they choose IaaS, PaaS, CaaS, or SaaS? Why?

Hints: consider the IT team size, maintenance burden, and how much control they need.

upf.

Outline

upf.

Putting It All Together

- In Session 1 we saw the **building blocks**: Virtual Switches, Virtual Routers, Virtual NICs, VLANs
- These components do not run in isolation; they are managed by **platforms**
- Platforms orchestrate **VMs** or **containers** and configure virtual networking under the hood

Two categories:

- **Cloud platforms**: manage infrastructure at scale
  - Compute, storage, networking, and **VMs**
- **Container orchestration platforms**: deploy and scale **containerized** applications

upf.

Cloud Platforms

- **Self-managed / Private Cloud**: you own or rent the servers, install and manage the virtualization software yourself. Full control, but requires dedicated staff and expertise.

Platform	Description	Logo
OpenStack	Open-source, widely used in private clouds	openstack.
VMware vSphere	Enterprise leader, proprietary	vmware
Proxmox VE	Open-source, lightweight alternative	PROXMOX

- **Managed / Public Cloud**: rent capacity on demand, pay-as-you-go. Provider handles hardware, networking, cooling, security, and updates.

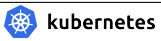
Provider	Description	Logo
Amazon Web Services	Market leader since 2006	aws
Microsoft Azure	Strong enterprise integration	Azure
Google Cloud Platform	Data and AI focus	Google Cloud

These map directly to the IaaS model we just covered: the provider (or you) manages compute, storage, and networking.

upf.

Container Orchestration Platforms

- **Self-hosted / Private**: you install and operate on your own servers

Platform	Description	Logo
Docker	Container runtime; works well on a single host Managing hundreds of containers across hosts? Unmanageable	
Kubernetes (K8s)	Created to solve this: scheduling, scaling, networking, self-healing	
OpenShift	Enterprise K8s by Red Hat; adds security, CI/CD, UI	

- **Hosted / Public**: cloud providers offer the same platforms as managed services. Each provider uses different names for essentially the same product (e.g., AWS ECS/EKS, Azure AKS, Google GKE, OpenShift on all major clouds).

Discussion: Platforms

A startup with 5 engineers needs to deploy
a web application that may go from 100 to 100,000 users.
Would you build a private cloud or use a public one? Why?

Hints: team size, upfront cost, time to market, scaling needs, control over infrastructure.

Outline

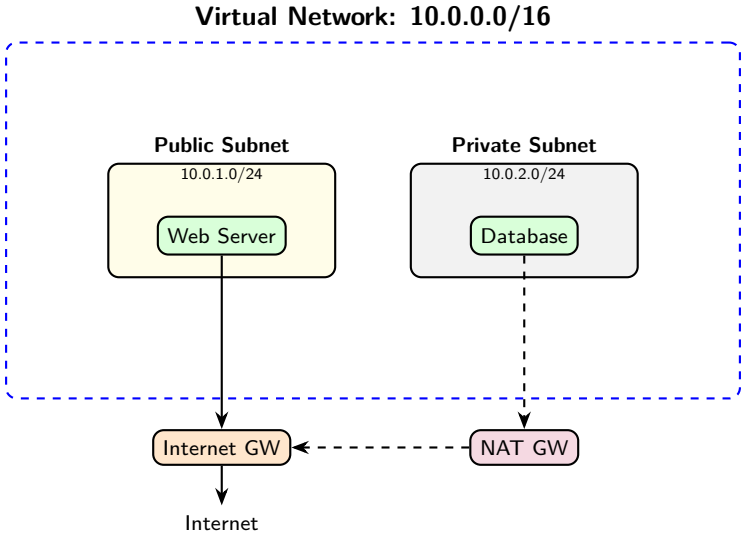
upf.

Virtual Networks in the Cloud

- Every cloud provider lets you create your own **isolated virtual network**
  - AWS: Virtual Private Cloud (VPC); Azure: Virtual Network (VNet); GCP: VPC Network
- Think of it as your **private data center network** in the cloud
- You define:
  - **Address space** (e.g., `10.0.0.0/16`)
  - **Subnets** (subdivisions of the address space)
  - **Routing rules** and **security policies**

Key: under the hood, cloud providers use **overlay networks** (covered in Block 5) to isolate tenants at scale.

Virtual Network Architecture



Subnets – Public vs Private

Public subnet:

- Has a route to an **Internet Gateway**
- Instances can have **public IP addresses**
- Accessible from the Internet (if security rules allow)
- Use for: web servers, load balancers, bastion hosts

Private subnet:

- **No direct route** to the Internet
- Instances only have **private IPs**
- Can access Internet outbound via **NAT Gateway**
- Use for: databases, internal services, application backends

upf.

Route Tables

- Each subnet is associated with a **route table**
- Route table = set of rules determining where traffic goes

Destination	Target	Subnet type
`10.0.0.0/16`	local (within virtual network)	Both
`0.0.0.0/0`	Internet Gateway	Public
`0.0.0.0/0`	NAT Gateway	Private

- **Local route**: traffic within the virtual network stays inside (always present)
- **Default route** (`0.0.0.0/0`): where to send everything else
- Public subnets → IGW; Private subnets → NAT GW

Internet Gateway vs NAT Gateway

Internet Gateway (the front door):

- **Bidirectional**: the world can reach your instance, and it can reach the world
- Instance needs a **public IP**
- Use for: web servers, load balancers, anything public-facing

NAT Gateway (the fire exit):

- **One-way**: your instance can reach the Internet (e.g., download updates, call APIs), but nobody outside can reach it directly
- Instance keeps a **private IP** only
- Use for: databases, backends that need to download patches but must stay hidden

Same **NAT concept from Block 3**, now as a managed cloud service.

upf.

Security Groups – Instance-Level Firewall

- **Security group** = virtual firewall attached to an instance
- Rules specify allowed **inbound** and **outbound** traffic
- **Stateful**: allow inbound → response automatically allowed

Direction	Protocol	Port	Source/Dest
Inbound	TCP	80	0.0.0.0/0
Inbound	TCP	443	0.0.0.0/0
Inbound	TCP	22	10.0.0.0/16
Outbound	All	All	0.0.0.0/0

Source: AWS, "Security Groups for Your VPC," docs.aws.amazon.com.

Network ACLs (Access Control Lists) – Subnet-Level Firewall

- **Network ACL** = firewall at the **subnet** level
- Rules evaluated in **order** (numbered, first match wins)
- **Stateless**: must explicitly allow both inbound AND outbound

Rule #	Direction	Protocol	Port	Action
100	Inbound	TCP	80	ALLOW
200	Inbound	TCP	443	ALLOW
*	Inbound	All	All	DENY

Source: AWS, "Network ACLs," docs.aws.amazon.com.

Security Groups vs Network ACLs

	Security Groups	Network ACLs
Scope	Instance level	Subnet level
State	Stateful	Stateless
Rules	Allow only	Allow + Deny
Evaluation	All rules evaluated	First match wins
Default	Deny all inbound	Allow all

Defense in depth: use both together.

- Security Groups → fine-grained per-instance control
- Network ACLs → broad subnet-level guardrails

Discussion: Cloud Networking

You deploy a web application in your cloud virtual network. The web server needs to be public, but the database must not be reachable from the Internet. How?

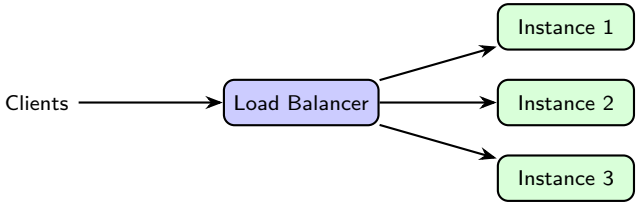
Hints: think about public vs private subnets, security groups, and NAT gateways.

Outline

upf.

Load Balancer

- Distributes incoming traffic across **multiple instances** (VMs or containers)
- Prevents any single instance from being overwhelmed
- If one instance fails, traffic is **redirected** to healthy ones

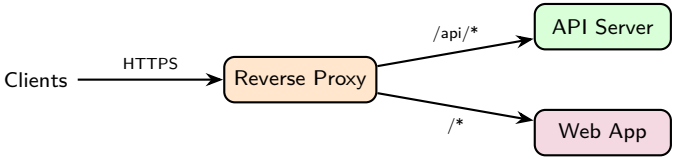


- **L4 (Transport Layer)** Load Balancer: routes based on IP and port (fast, simple)
- **L7 (Application Layer)** Load Balancer: routes based on HTTP headers, URL path, cookies (smarter)
- Cloud examples: AWS ALB/NLB, Azure Load Balancer, GCP Cloud Load Balancing

Source: AWS, "Elastic Load Balancing Documentation," docs.aws.amazon.com.

Reverse Proxy

- Sits **in front of** backend servers, receives all client requests
- Clients never communicate directly with the backend



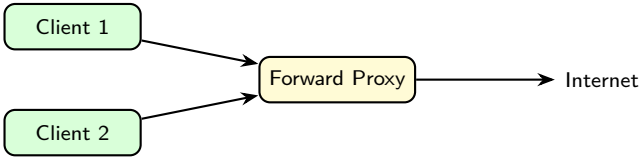
Key functions:

- **TLS (Transport Layer Security) termination**: handles HTTPS encryption, backends receive plain HTTP
- **URL routing**: `/api/*` → API server, `/*` → web app
- **Caching**: stores responses to reduce backend load
- Examples: NGINX, HAProxy, AWS CloudFront, Azure Front Door

upf.

Forward Proxy

- Sits **in front of clients**, controls their **outbound** traffic
- Clients send requests to the proxy, which forwards them to the Internet



Key functions:

- **Access control**: block certain websites or domains
- **Caching**: store frequently accessed content, reduce bandwidth
- **Anonymity**: external servers see the proxy's IP, not the client's
- **Logging**: monitor what employees or VMs access
- Examples: Squid, corporate firewalls with proxy mode

Reverse Proxy vs Forward Proxy

	Forward Proxy	**Reverse Proxy**
Protects	Clients (outbound)	Servers (inbound)
Position	In front of clients	In front of servers
Who knows?	Server sees proxy, not client	Client sees proxy, not server
Use case	Control outbound access	TLS termination, routing, caching

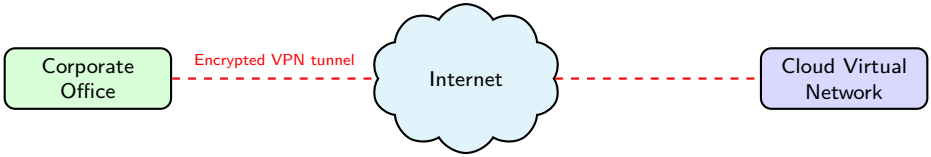
Think of it this way:

- **Forward proxy**: "I control what my users can access outside"
- **Reverse proxy**: "I control how outside users reach my servers"

upf.

VPN – Virtual Private Network

- A **VPN (Virtual Private Network)** creates a **secure, encrypted tunnel** over the public Internet
- Connects remote networks or users as if they were on the same private network



Two main types:

- **Site-to-site VPN**: connects two networks (e.g. office ↔ cloud VPC)
- **Client VPN**: individual user connects to a remote network (e.g. employee working from home)
- Cloud examples: AWS VPN, Azure VPN Gateway, GCP Cloud VPN

Discussion: Cloud Network Services

A company has a web application with a public frontend, a private API, and a database. They also need employees to access the cloud from the office. Which services would you use and where?

Hints: load balancer for the frontend, reverse proxy for routing, VPN for office access, private subnet for the database.

Outline

upf.

Key Takeaways

1. Virtualization is the **technology**; cloud is the **business model**
2. Five NIST characteristics define true cloud computing
3. **IaaS / PaaS / CaaS / SaaS** differ in who manages what
4. Cloud and container **platforms** deliver these models at scale
5. A **virtual network** is your isolated cloud network (CIDR, subnets, route tables)
6. **Security groups** (instance) + **ACLs** (subnet) = defense in depth
7. **Load balancers**, **reverse/forward proxies**, and **VPNs** are essential cloud network services

Discussion

You are designing a cloud network for a company with public web servers and private databases. Which components would you use and why?

Think about: public vs private subnets, security groups, NAT gateways, load balancers, VPN.

References

1. NIST SP 800-145, Mell & Grance, “The NIST Definition of Cloud Computing,” 2011.
2. NIST SP 800-144, “Guidelines on Security and Privacy in Public Cloud Computing,” 2011.
3. AWS, “Amazon VPC User Guide,” docs.aws.amazon.com.
4. Microsoft Azure, “Azure Virtual Network Documentation,” docs.microsoft.com.
5. Google Cloud, “VPC Documentation,” cloud.google.com/vpc/docs.
6. CSA, “Security Guidance for Critical Areas of Focus in Cloud Computing v5,” 2022.
7. Erl, Puttini & Mahmood, *Cloud Computing: Concepts, Technology & Architecture*, Prentice Hall, 2013.
8. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
9. AWS, “Elastic Load Balancing Documentation,” docs.aws.amazon.com.
10. NGINX, Inc., “What Is a Reverse Proxy?,” nginx.com/resources.
11. AWS, “AWS VPN Documentation,” docs.aws.amazon.com.